



Agents with Ollama and LangGraph

Ollama and LangGraph

- Ollama
- Agents
- Architecture of agents
- LangChain Stack

Ollama

Agents with Ollama and LangGraph

Llama



- Llama (Large Language Model Meta AI) is a family of models introduced by Meta AI, starting in 2023.
 - <https://huggingface.co/meta-llama>
- All autoregressive architectures, decoders only, trained on open datasets
- All models are openly available
 - LLaMA (February '23) à 7B, 13B, 32B, 65B
 - Llama 2 (July '23) à 7B, 13B, 70B
 - Llama 3 (April '24) à 8B, 70B
 - 3.1 (July '24) à 8B, 70B, 405B
 - 3.2 (September '24) à 1B, 3B, 11B, 90B (with multimodal version)
 - Plus other versions (e.g. Code Llama - based on Llama 2, or models with instructions)

Ollama

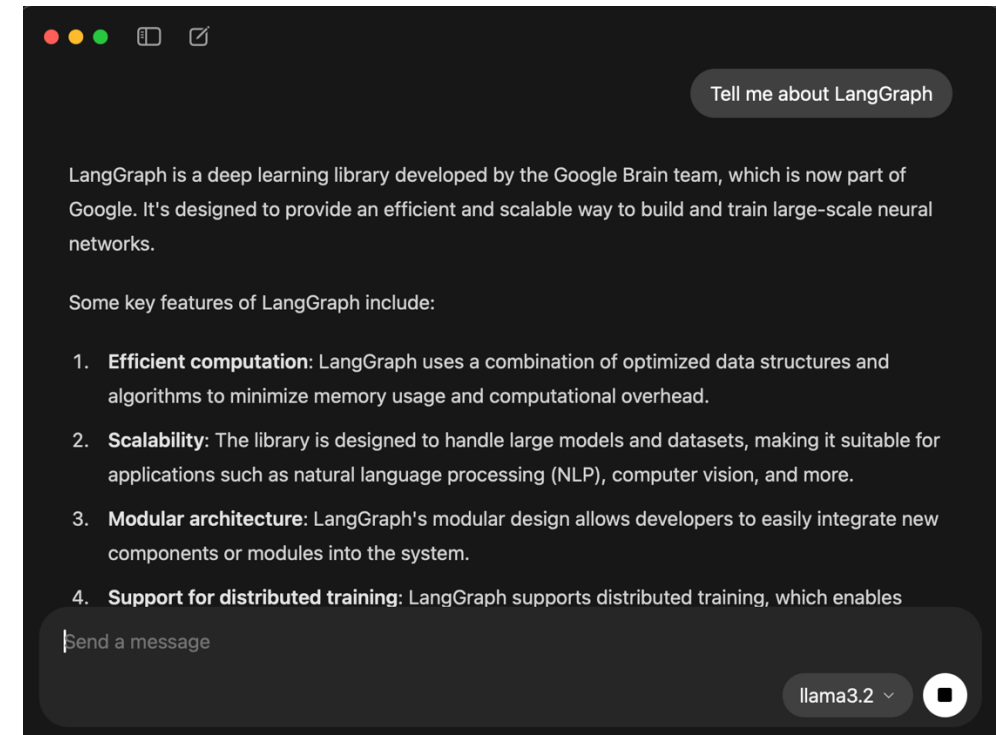
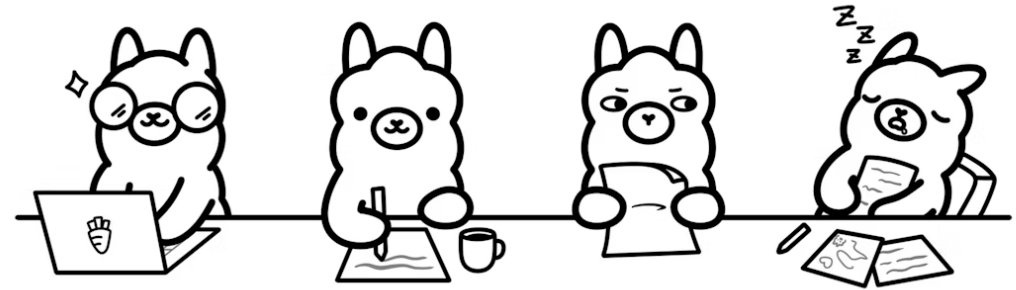
- The easiest way to get up and running with large language models such as gpt-oss, Gemma 3, DeepSeek-R1, Qwen3 and more
- To get started, download Ollama on macOS, Windows or Linux

```
ollama run gemma3
```

Multiline input

- For multiline input, you can wrap text with """:

```
>>> """Hello,  
... world!  
...  
I'm a basic program that prints the  
famous "Hello, world!" message to the  
console.
```



Ollama - Python

- Start by downloading a model:

```
ollama run gemma3
```

- Then install Ollama's Python library:

```
pip install ollama
```

- Lastly, chat with the model:

```
from ollama import chat
from ollama import ChatResponse

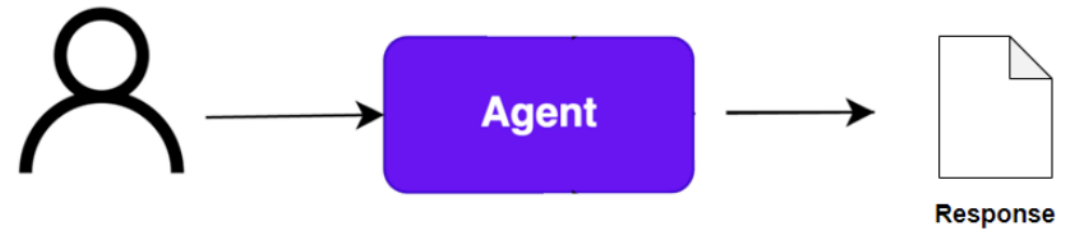
response: ChatResponse =
chat(model='gemma3', messages=[
    {
        'role': 'user',
        'content': 'Why is the sky blue?',
    },
])
print(response['message']['content'])
# or access fields directly from the
# response object
print(response.message.content)
```

Agents

Agents with Ollama and LangGraph

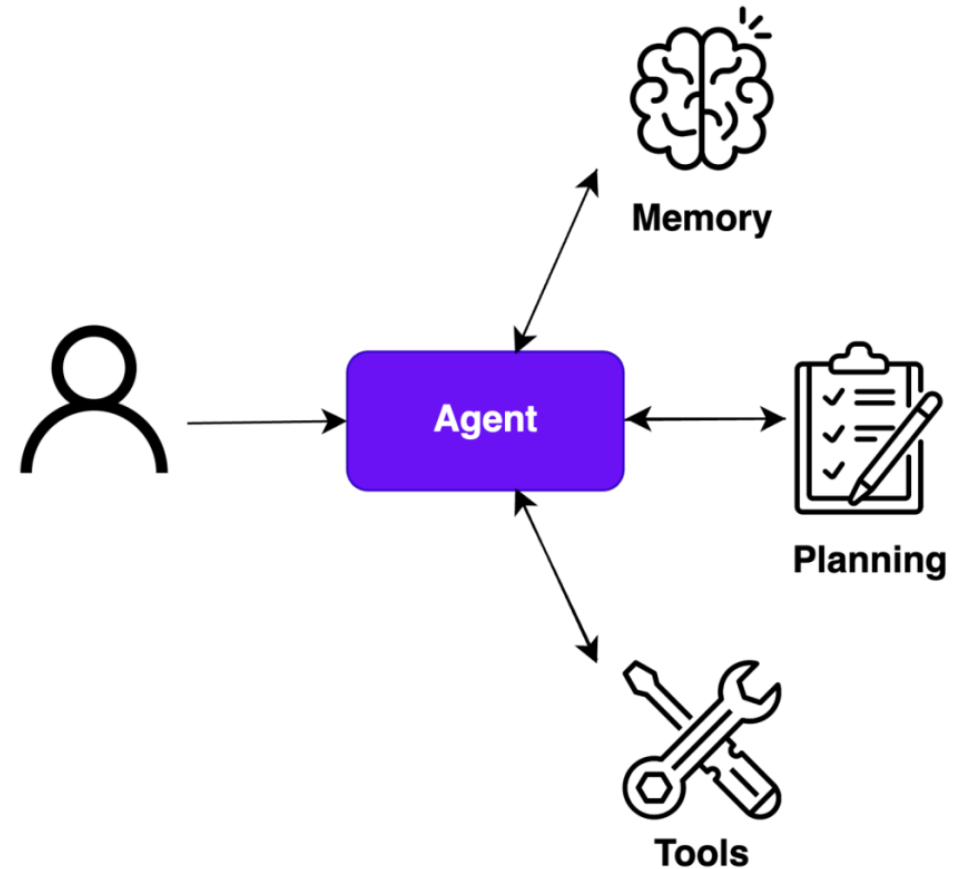
Traditional approach

- In traditional LLM applications, the model follows a predetermined control flow—a fixed sequence of steps before and after LLM calls.
 - a Retrieval-Augmented Generation (RAG) system retrieves relevant documents in response to a query and passes them to the LLM to base its response on. While effective, this approach limits the model to a static workflow, potentially restricting its problem-solving capabilities.



Agentic

- Agentic behaviour refers to the ability of an LLM to decide its own control flow in order to solve more complex problems
- The model becomes an "agent" that can decide what steps to take, what tools to use, and when to end a process
- This flexibility enables more sophisticated applications, allowing the LLM to handle tasks that require dynamic decisions and adaptability



Advantages

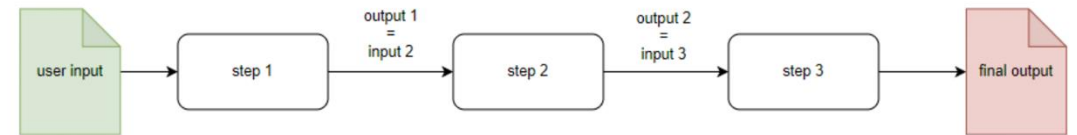
- Improved problem solving
 - can tackle more complex tasks by choosing the most appropriate actions
- Dynamic control flow
 - are not limited to a fixed sequence of steps, allowing for more flexible and efficient processes
- Improved efficiency
 - by deciding when sufficient information has been gathered, unnecessary calculations can be reduced.

LLM Chains

- An LLM chain is a series of requests and operations that guide an LLM through a sequence of tasks, enabling more complex and nuanced AI interactions
 - provides a structured workflow for artificial intelligence to perform sophisticated tasks
- For example, a simple LLM chain for summarising a long article could involve the following steps:
 - Ask the LLM to read the article
 - Ask to identify the key points
 - Request a concise summary based on these points
 - Refine the summary for clarity and consistency

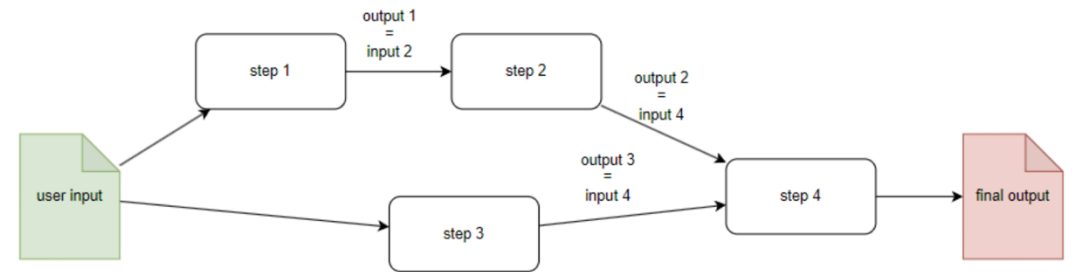
Sequential chain

- Consists of steps in which each step takes an input and produces an output
- The output of one step becomes the input of the next step
- Effective when it comes to sub-chains designed for individual entrances and exits
- Ensures a smooth and continuous flow of information, in which each stage seamlessly passes its output to the next stage



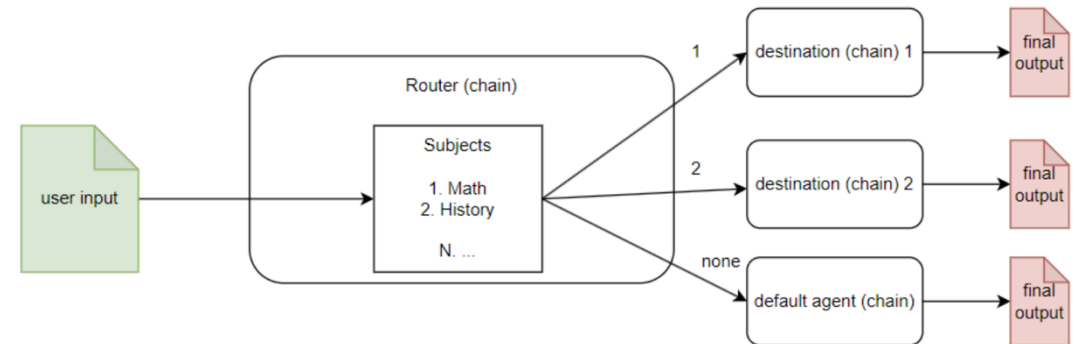
Tree Chain

- Not all sequential chains work with a single input and output string
- In more complex configurations, these chains manage multiple inputs and generate multiple final outputs.



Router Chain

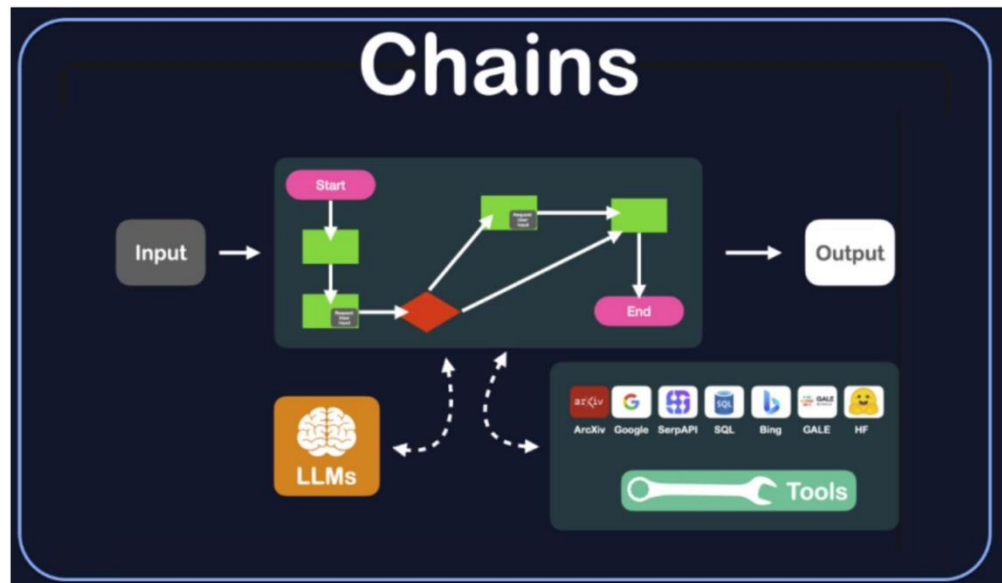
- Used for complicated tasks
- Multiple sub-chains, each specialized for a particular type of input
- A router chain that decides which subchain to pass the input to
- It consists of:
 - **Router chain**: responsible for selecting the next chain to call
 - **Destination chains**: chains to which the router chain can direct traffic
 - **Default chain**: used when the router cannot decide which subchain to use



From chains to agents

- An AI system with agents is capable of setting its own goals, planning actions to achieve them, and making independent decisions based on its understanding of the environment and the task at hand
- Unlike reactive artificial intelligence, which merely responds to inputs, agentic artificial intelligence can take initiative, adapt to changing circumstances, and even learn from its own experiences
- LLM chains serve as building blocks for agentic behaviour, enabling AI to break down complex tasks, reason about multi-step problems, and interact with the environment in more nuanced ways
- To illustrate the difference, let's consider a virtual assistant:
 - A responsive AI assistant could answer the question "What's the weather like?" with an up-to-date weather report
 - An agentic artificial intelligence assistant, after hearing the same question, could not only provide the weather forecast, but also suggest appropriate clothing for the day, recommend indoor or outdoor activities based on the forecast, and even offer to set reminders for weather-dependent activities

From chains to agents

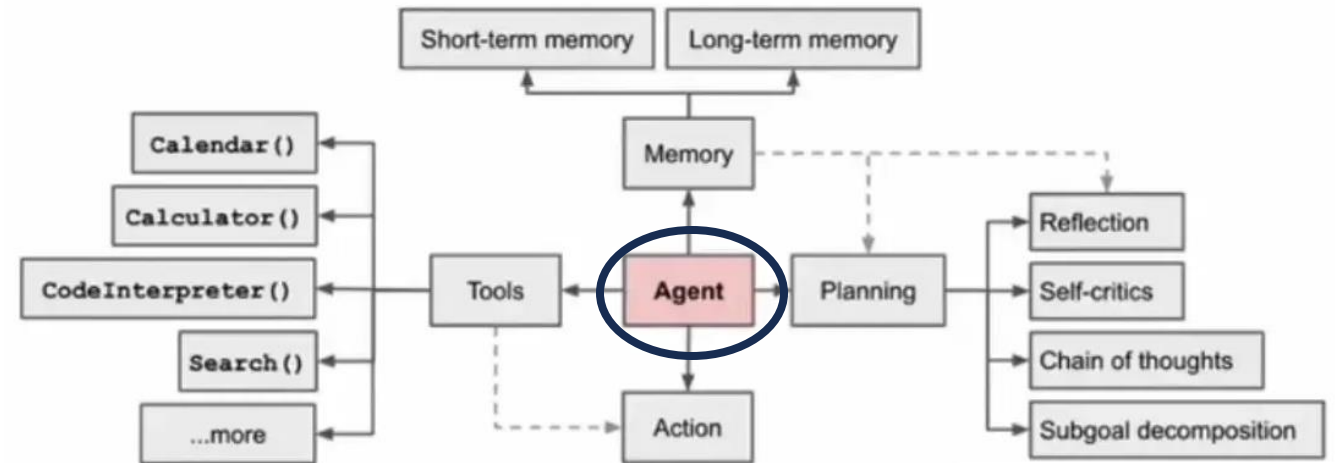


Architecture of agents

Agents with Ollama and LangGraph

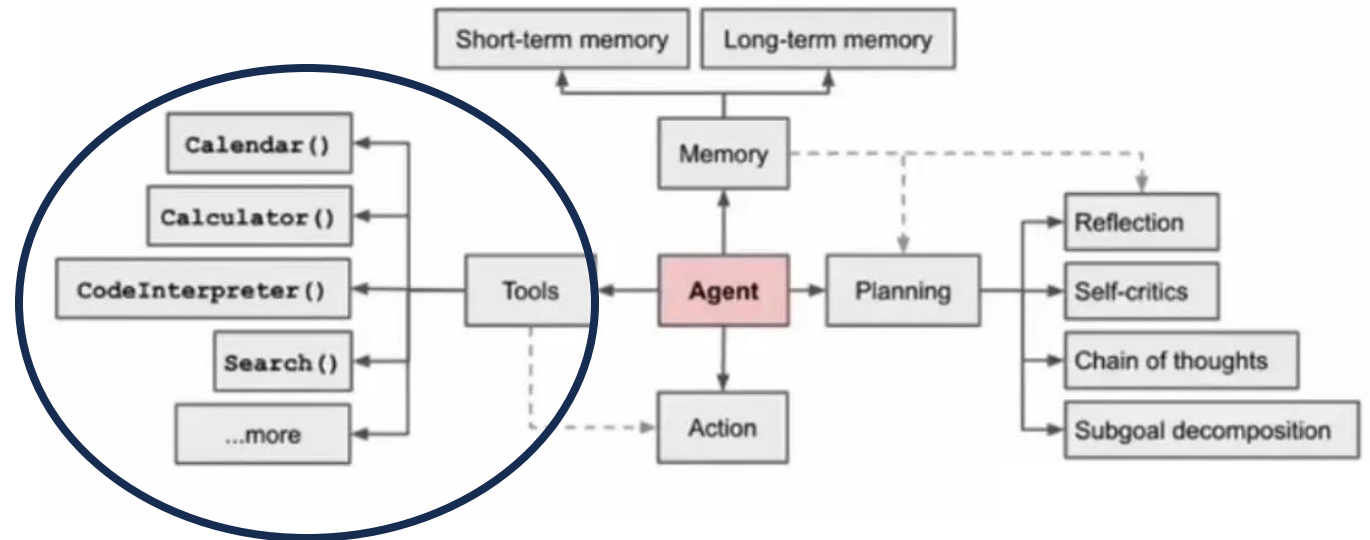
Agent

- The agent is the central decision-making entity that orchestrates the use of tools, memory, and planning. It determines how to approach a given task, selecting the most appropriate tool or planning strategy based on the complexity of the problem.



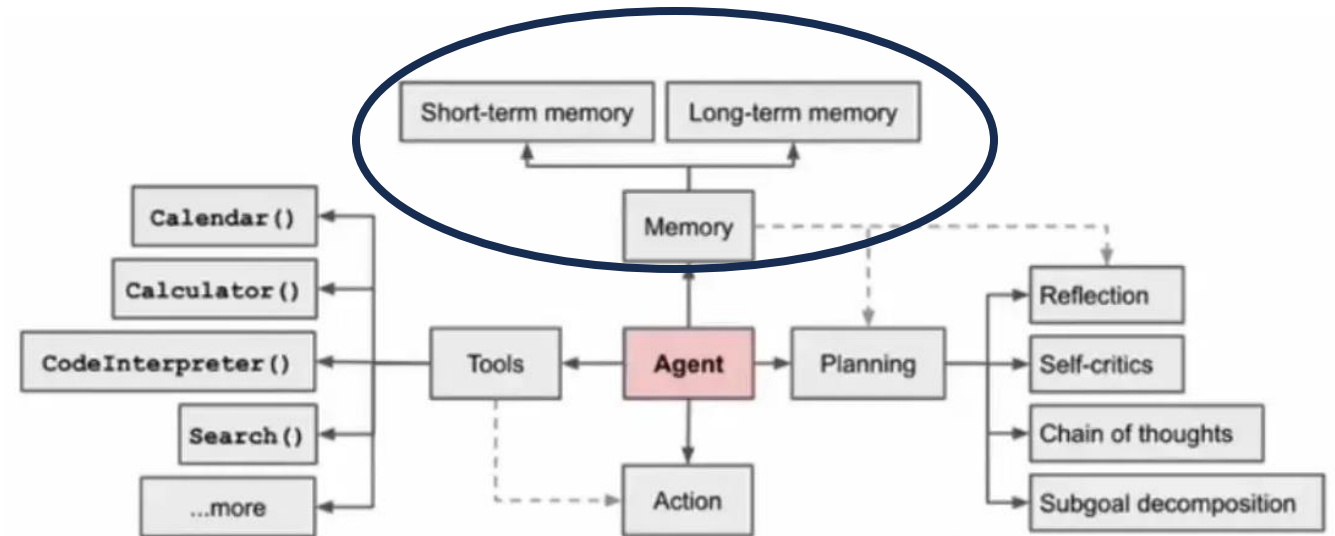
Tools

- Tools are specialised extensions that expand the agent's capabilities. Examples include:
 - the Calendar, which helps schedule events or keep track of deadlines;
 - the Calculator, used for precise mathematical operations;
 - the CodeInterpreter, which allows you to perform coding or data analysis tasks
 - the Search tool, which retrieves information in real time from external sources such as the Internet.
- For example, if tasked with planning the timing of a project, the agent could use the Calendar to assign dates and the Calculator to optimise resource allocation.



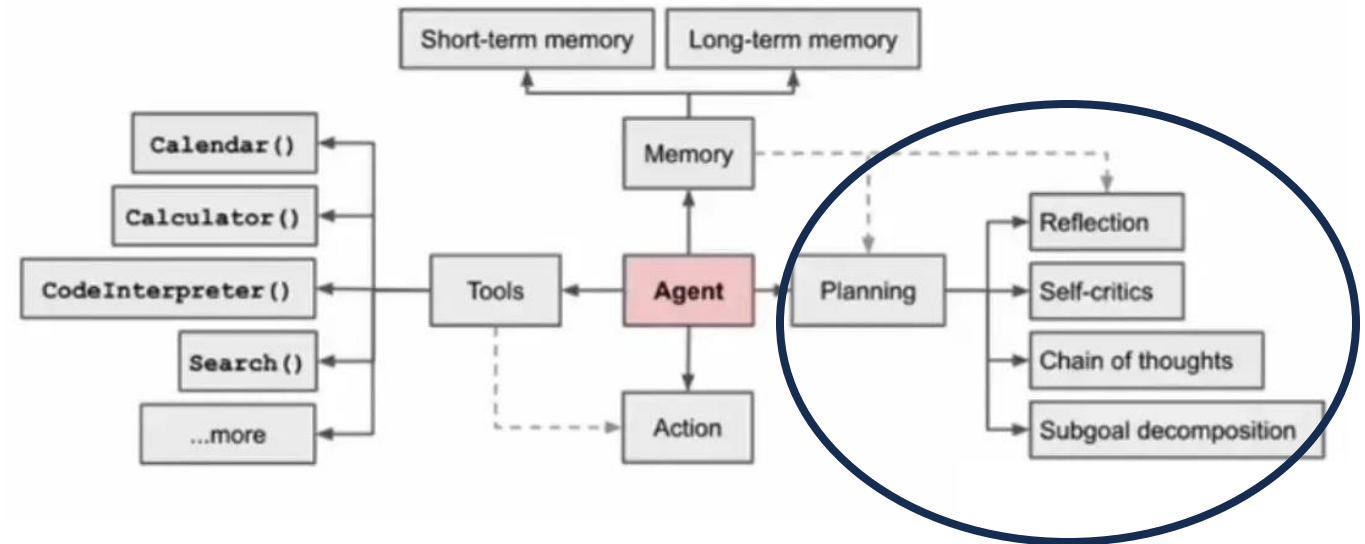
Memory

- Memory is divided into short-term memory and long-term memory.
- Short-term memory stores temporary information, such as recent instructions or intermediate results, while long-term memory stores persistent knowledge for future reference.
- For example, the agent could use short-term memory to temporarily store search results or the context of a conversation, and long-term memory to store user preferences, enabling personalised assistance over time.



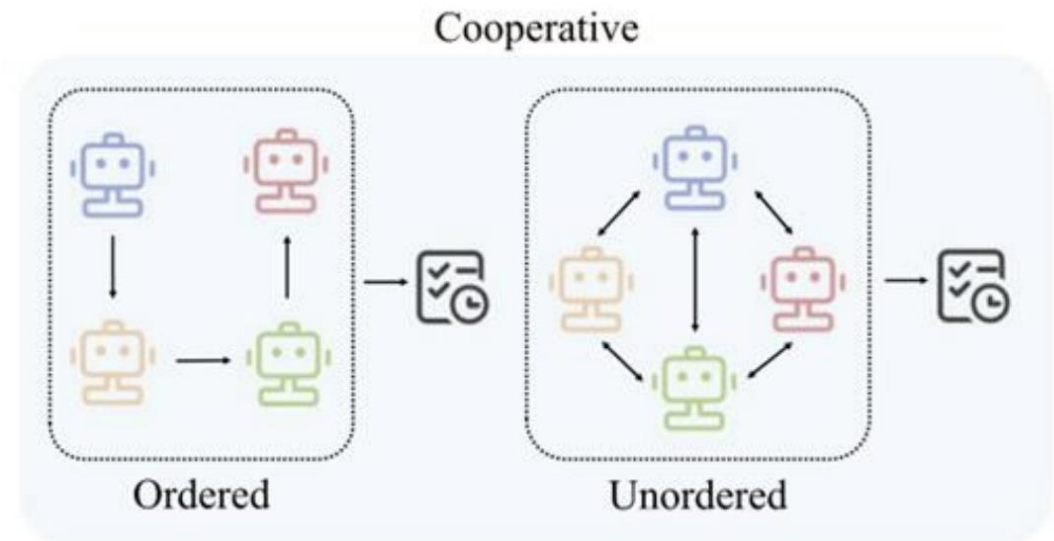
Planning

- The planning module enables the agent to devise strategies for solving problems. This involves using or combining different strategies.
 - Reflection
 - Self-critics
 - Chain-of-Thoughts
 - Subgoal decomposition



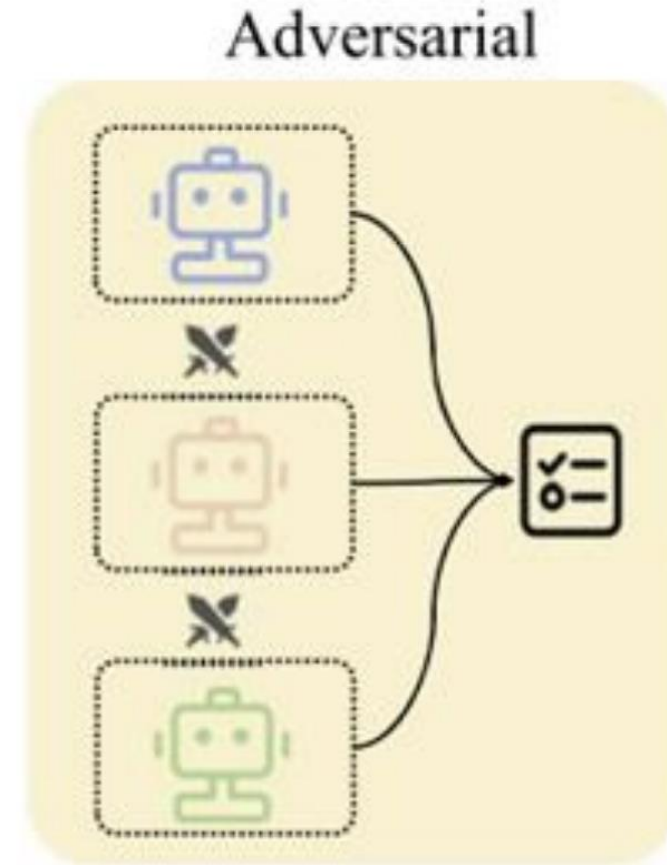
Interaction patterns: cooperative

- Agents work together to achieve a common goal
- The basic process includes
 - definition of goals
 - the breakdown of tasks
 - information sharing
 - collaborative decision-making
 - feedback on performance
- Process: the agents
 - first establish common objectives based on the requirements of the task
 - break down complex tasks into multiple subtasks assigned to different agents.
 - share information
 - make joint decisions through communication and negotiation to reach a consensus
- During the execution of the task, each agent
 - performs activities according to their respective roles
 - provides feedback to adjust strategies and optimise the execution process



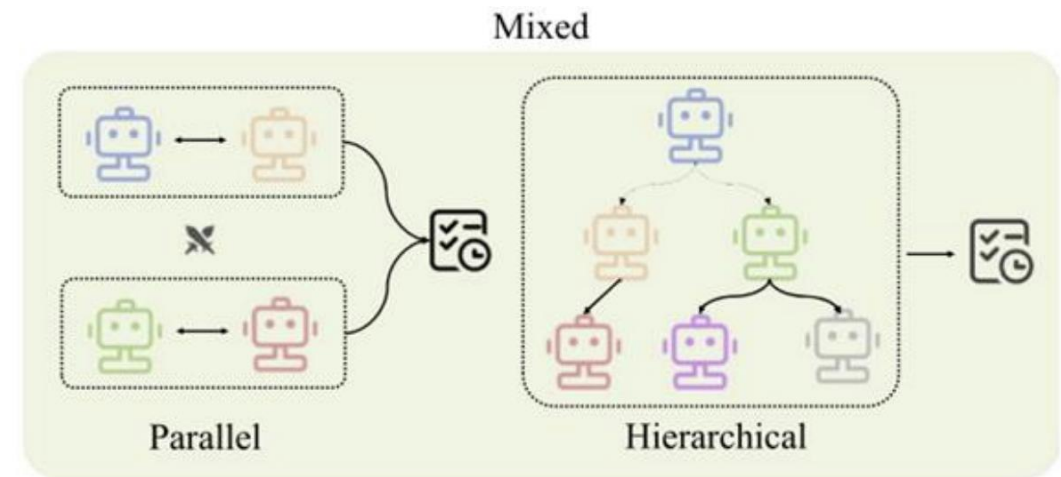
Interaction patterns: competitive

- Every agent pursues the maximization of their own interests
- The basic process includes
 - definition of goals
 - the formulation of the strategy
 - interactive games
 - evaluation of results
- Process: the agents
 - first establish objectives to maximise their interests
 - formulate competitive strategies based on the behaviour of opponents
 - in the interaction phase of the game, they implement strategies through interactions to try to obtain maximum benefits
 - evaluate the results of the game and adapt strategies to cope with future competition

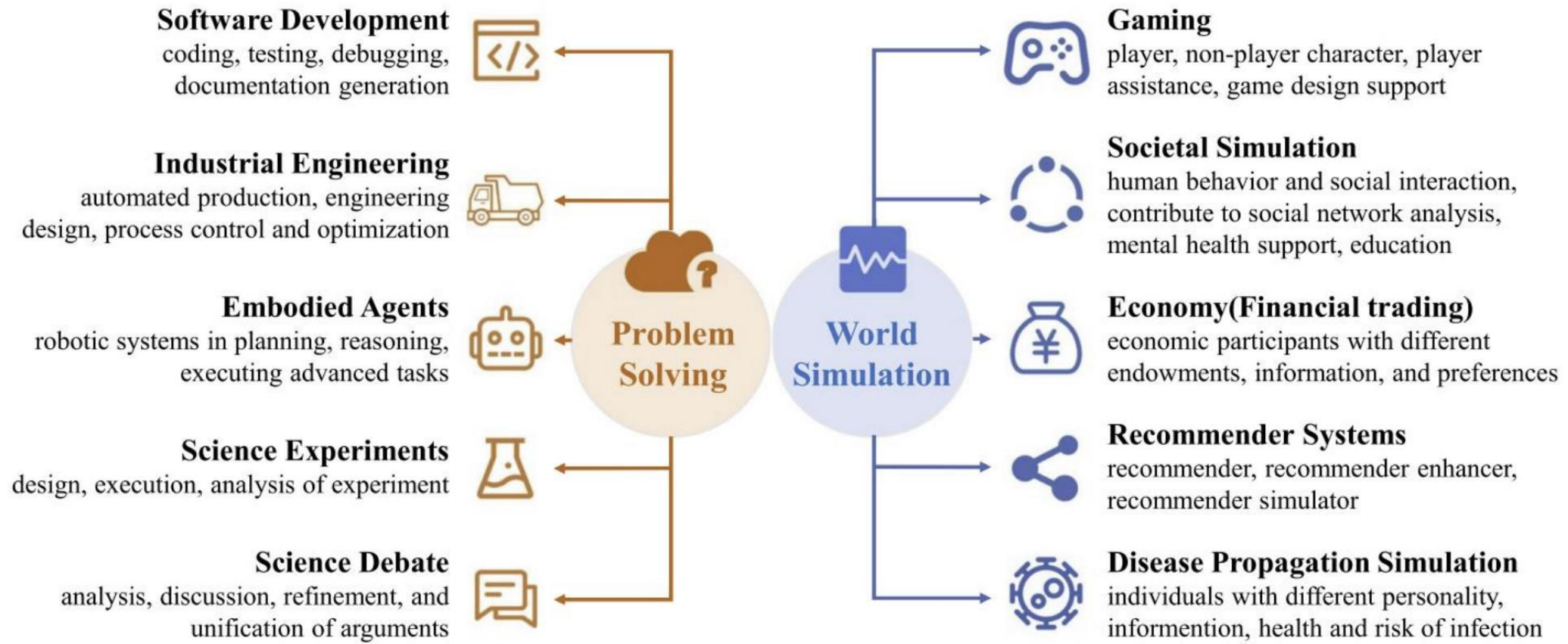


Interaction patterns: mix

- They combine characteristics of both cooperative and adversarial interaction.
- Types of interaction
 - Parallel
 - agents collaborate independently on separate tasks
 - share some information without interfering with each other
 - a form of competition is then applied
 - Hierarchical
- Relationships between agents typically manifest themselves as a tree structure
 - parent node
 - sets overall goals
 - breaks down tasks
 - assign tasks to agents of the child node
 - child node
 - perform specific tasks
 - provide feedback on performance



Application of multi-agent systems



Overview

- Open-source
- It focuses heavily on collaboration between multiple agents
- Code-based approach with 'roles' for each agent
- Provides a robust environment for task orchestration

Key features

- Role-based architecture: Specialised roles are defined with objectives, knowledge, etc.
- Shared memory system: Agents can store and retrieve data from a common memory.
- Supports advanced tools: If you wish to integrate a third-party API, you can set up a 'tool' for this purpose
- Error handling: Automatic guardrails to prevent incorrect tasks

Pros and cons

- Pros: Excellent for multi-stage activities, promotes synergy between agents, flexible
- Cons: requires some technical expertise; lacks advanced security or default encryption.

Use cases

- Project management: Division of tasks among several agents
- Financial analysis: Agents handle risk analysis, data cleansing, etc.
- SEO content: Agents handle research, drafting and editing.

Ideal for

- Small to medium-sized development teams seeking a multi-agent system with a strong open-source community

Overview

- Supported by Microsoft
- Open-source solution geared towards enterprise-level tasks
- Use LLM to orchestrate multi-agent flows
- Ability to create multiple agents

Key features

- Deep integration of LLM: GPT-4, GPT-3.5, Claude, PaLM, etc.
- Human supervision: For sensitive tasks, partial or final manual checks may be performed.
- Code generation: Agents can produce code, test it, and then refine it.

Pros and cons

- Pros: Strong corporate atmosphere, advanced debug logs, excellent with GPT-4
- Cons: Requires a lot of resources (expensive if usage is high), some rough patches for large-scale competition
- Scalability: Careful architecture design is required if you have thousands of tasks.

Use cases

- Software dev: agents who write or improve code
- Data analysis: Agents for data ingestion, cleaning, and synthesis
- Customer support: Agents to create or refine knowledge base entries

Ideal for

- Companies seeking robust multi-agent solutions that leverage advanced LLMs, with moderate to high budgets



Overview

- Google Cloud
- Code-free approach to building LLM-driven agents
- User-friendly web interface
- You can define tasks, train them or link them to data, and watch them perform in real time

Key features

- Drag-and-drop environment
- Predefined modules: search, chat, or data recovery
- Integrated compliance: HIPAA or financial compliance for sensitive data.
- Integration: Connects well with BigQuery, Dataflow, etc.

Pros and cons

- Pros: Easy to configure, suitable for businesses, advanced data features
- Cons: May be expensive if usage is high or if you want advanced custom logic

Use cases

- Chatbot for customer service with knowledge of company documents
- Automation of workflows for repetitive tasks, such as labelling or data classification
- Processing of health or financial data.

Ideal for

- Large organisations or small teams that prefer no-code
- People already familiar with Google Cloud

Overview

- No-code AI platform
- Helps you quickly build simpler AI flows
- More limited in advanced multi-agent logic
- Suitable for automations that would otherwise require a developer

Key features

- No-code: Perfect for non-developers
- Data variety: Supports text, images, and audio
- Marketplace for pre-trained models: for example, you can choose a pre-trained sentiment or classification model to speed up your pipeline

Pros and cons

- Pros: Fast, easy to use, free trial plan.
- Cons: Does not easily handle highly specialised tasks. May not be able to handle a huge amount of data.

Use cases

- Simple classification or chatbot for your e-commerce site.
- Automatic scanning of emails and documents for small offices.
- Start-ups that do not have development resources but want artificial intelligence automation.

Ideal for

- Beginners or small businesses with standard AI tasks

LangChain Stack

Agents with Ollama and LangGraph

LangChain Stack



Overview

- Open-source library
- Possibility of linking steps or "modules" for advanced artificial intelligence tasks.
- It is not purely an 'agent platform'.
- It has features for building custom agents or orchestrating multi-step workflows.

Key features

- Chains: Link components such as prompt templates, memory, or logic modules together.
- Open ecosystem: Many expansions built by the community.
- Memory: Offers short or long-term memory, allowing the agent to 'remember' previous steps.

Pros and cons

- Pros: Very flexible and developer-friendly, rapidly evolving community.
- Cons: Some describe it as 'always evolving', so features may break and are not certified at an enterprise level.
- Performance: Strong competition can be a problem.

Use cases

- Document-based tasks: Summaries, questions and answers, data extraction.
- Writing pipeline: from research to editing.
- Classification or transformation of data with partial automation.

Ideal for

- Startups or development teams who enjoy building custom workflows and are comfortable modifying open-source code

LangChain

- Easiest way to start building with LLMs in under 10 lines of code
 - OpenAI
 - Anthropic
 - Google
 - and more...
- Agents are built on top of LangGraph in order to provide
 - durable execution
 - streaming
 - human-in-the-loop
 - persistence
- You do not need to know LangGraph for basic LangChain agent usage.

Create an agent

```
# pip install -qU "langchain[anthropic]" to call the model

from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="anthropic:claude-3-7-sonnet-latest",
    tools=[get_weather],
    prompt="You are a helpful assistant",
)

# Run the agent
agent.invoke(
    {"messages": [{"role": "user", "content": "what is the weather in sf"}]}
)
```

LangGraph

- Very low-level and focused entirely on **agent orchestration**.
- Commonly use LangChain components
 - but you don't need to use LangChain to use LangGraph

```
from langgraph.graph import StateGraph, MessagesState, START, END

def mock_llm(state: MessagesState):
    return {"messages": [{"role": "ai", "content": "hello world"}]}

graph = StateGraph(MessagesState)
graph.add_node(mock_llm)
graph.add_edge(START, "mock_llm")
graph.add_edge("mock_llm", END)
graph = graph.compile()

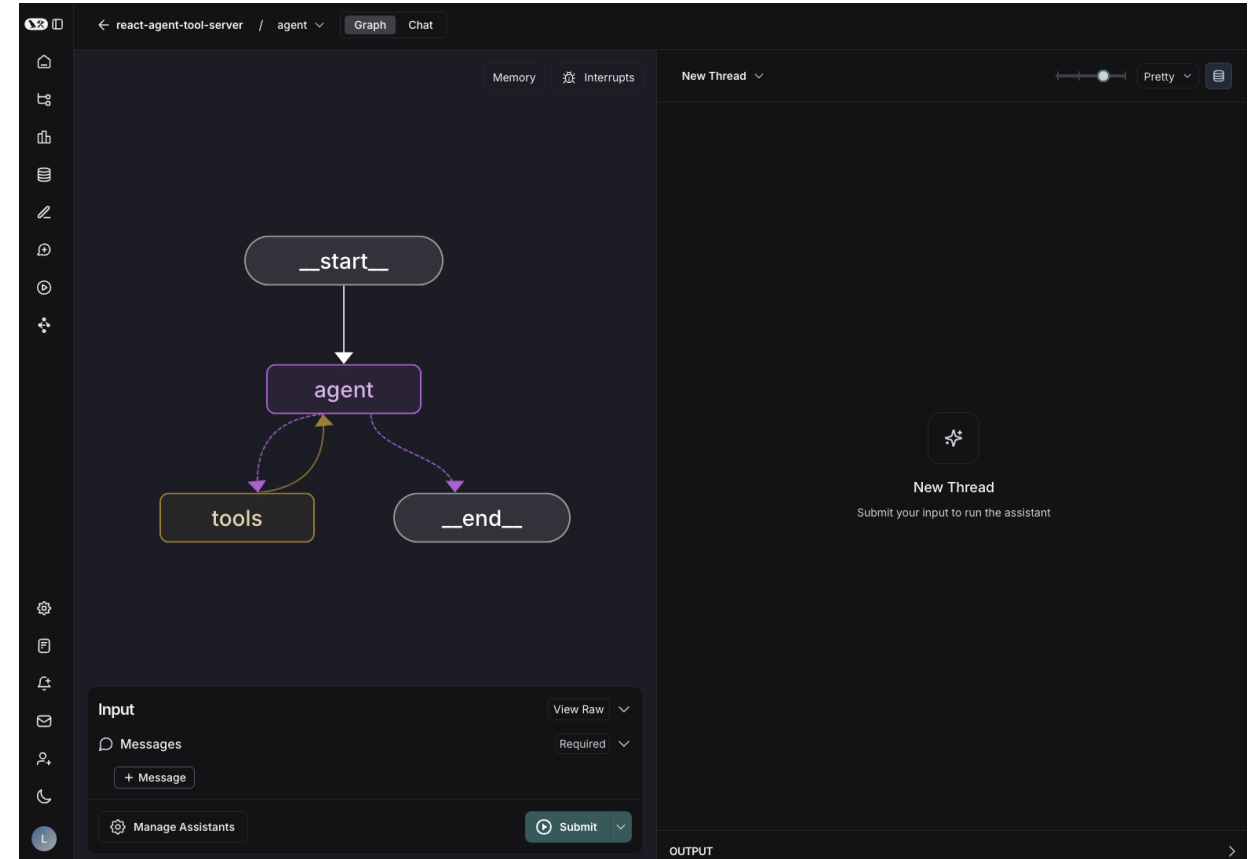
graph.invoke({"messages": [{"role": "user", "content": "hi!"}]})
```

LangGraph Studio

- Specialized agent IDE that enables visualization, interaction, and debugging of agentic systems that implement the LangGraph Server API protocol.
- Integrated with LangSmith to enable tracing, evaluation, and prompt engineering

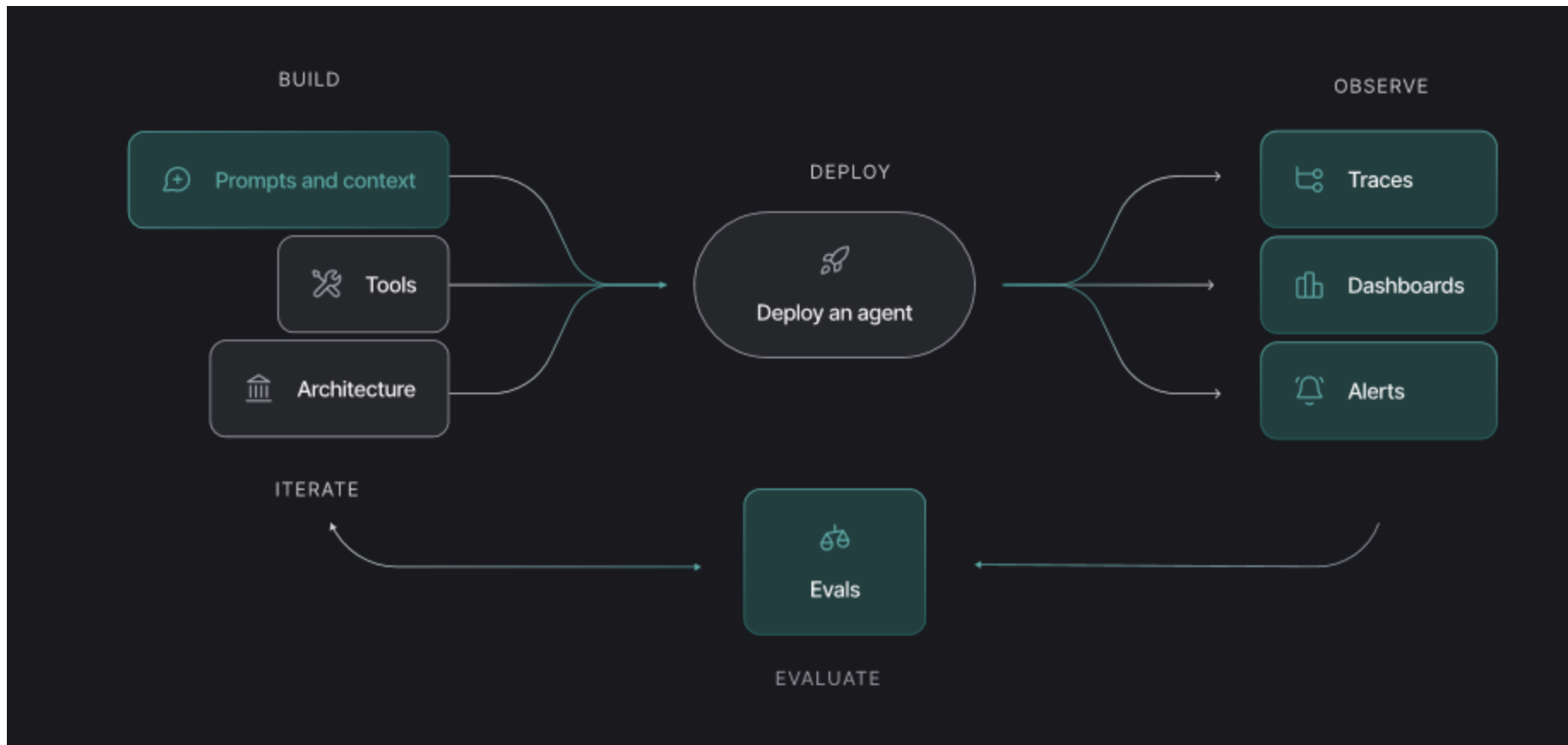
Key features:

- Visualize your graph architecture
- Run and interact with your agent
- Manage assistants
- Manage threads
- Iterate on prompts
- Run experiments over a dataset
- Manage long term memory
- Debug agent state via time travel



LangSmith

- Platform for building production-grade LLM applications. Monitor and evaluate your application, so you can ship quickly and with confidence.



- Observability

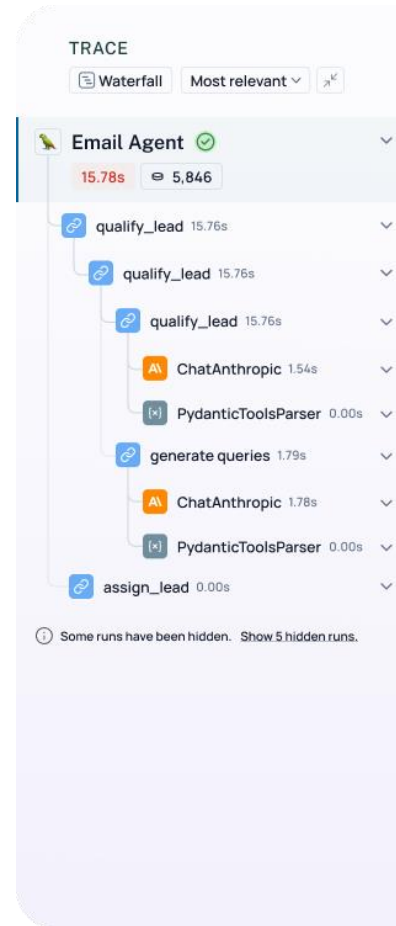
- **View traces:** Access and manage your traces through the UI and API, including filtering, exporting, sharing, and comparing traces.
- **Monitoring:** Set up dashboards and alerts to monitor your application performance and receive notifications when issues arise.
- **Automations:** Configure rules, webhooks, and online evaluations to automate your observability workflows.
- **Human Feedback:** Collect and manage human feedback on your application outputs through annotation queues and inline annotation.
- **Trace a RAG application**

- Evaluation

- **Datasets:** Create and manage datasets for evaluation, including creating datasets through the UI or SDK and managing existing datasets.
- **Evaluations:** Run evaluations on your applications using various methods and techniques, including different evaluator types and evaluation techniques.
- **Analyze experiment results:** View and analyze your evaluation results, including comparing experiments, filtering results, and downloading data.
- **Annotation & human feedback:** Collect human feedback on your application outputs through annotation queues and inline annotation.

LangSmith

- Prompt engineering
 - **Create and update prompts:** Create prompts through the UI or SDK, configure settings, use tools, include multimodal content, and connect to different model providers.
 - **Manage prompts:** Organize your prompts with tags, commit changes, trigger webhooks, and share prompts through the public prompt hub.
 - **Prompt hub:** Access and manage prompt tags and explore the LangChain Hub for community prompts.
 - **Prompt playground:** Test and experiment with prompts using custom endpoints and model configurations.



Email Agent

Success 15.78s 5,846

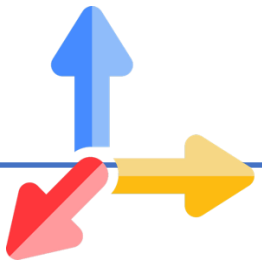
Input

Company
PollyCorp
Employee Info
Email polly@pollycorp.com
Last Name Parrot
Name Polly
Title Senior Strategy Manager
Lead Source
null

Output

Company
PollyCorp
Lead Decision
do_not_pursue
Lead Explanation
While PollyCorp is a large organization with a broad presence, there...

Tavily



- Search engine optimized for LLMs, aimed at efficient, quick and persistent search results.
- Unlike other search APIs such as Serp or Google, it focuses on optimizing search for AI developers and autonomous AI agents
 - aggregates up to 20 sites per a single API call
 - uses proprietary AI to score, filter and rank the top most relevant sources and content to your task, query or goal.
- Allows developers to add custom fields such as context and limit response tokens to enable the optimal search experience for LLMs
- **1,000 free API Credits every month**

```
# pip install tavily-python

from tavily import TavilyClient

tavily_client = TavilyClient(api_key="tvly-
YOUR_API_KEY")
response = tavily_client.search("Who is Leo
Messi?")

print(response)
```